# Floating-Point Operator v3.0



## Introduction

The Xilinx Floating-Point core provides designers with the means to perform floating-point arithmetic on an FPGA. The core can be customized to allow optimization for operation, wordlength, latency, and interface.

***Acknowledgement***

QinetiQ

*This Xilinx core is based on IP originally licensed from QinetiQ Ltd.*

## Features

- Available for Virtex™-II, Virtex-II Pro, Virtex-4, Virtex-5, Spartan™-II, Spartan-3, and Spartan-3E FPGA family members
- Supported operators:
  - multiply
  - add/subtract
  - divide
  - square-root
  - compare
  - conversion from floating-point to fixed-point
  - conversion from fixed-point to floating-point
  - conversion between floating-point types
- Compliance with *IEEE-754 Standard* (with only minor documented deviations)
- Support for DSP48 on Virtex-4 FPGAs and DSP48E on Virtex-5 FPGAs
- Parameterized fraction and exponent wordlengths
- Optimizations for speed and latency
- Fully synchronous design using a single clock
- For use with the CORE Generator™ which is available in the Xilinx ISE™ v8.2i.

## Overview

The Xilinx Floating-Point core allows a range of floating-point arithmetic operations to be performed on FPGAs. The operation is specified when the core is generated, and each variant has a common interface. This interface is shown in Figure 1. When a user selects an operation that requires only one operand, the B input is omitted.

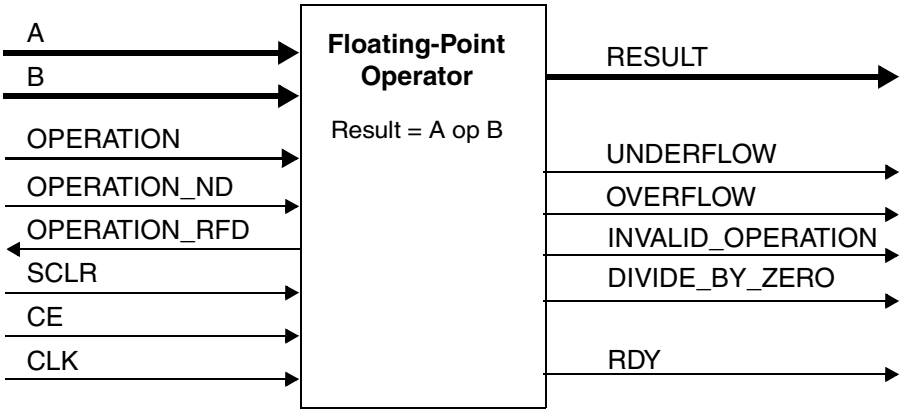


*Figure 1:* **Block Diagram of Generic Floating-Point Binary Operator Core**

© 2006 Xilinx, Inc. All rights reserved. XILINX, the Xilinx logo, and other designated brands included herein are trademarks of Xilinx, Inc. The QinetiQ logo is a trademark of QinetiQ Ltd. All other trademarks are the property of their respective owners. Xilinx is providing this design, code, or information "as is." By providing the design, code, or information as one possible implementation of this feature, application, or standard, Xilinx makes no representation that this implementation is free from any claims of infringement. You are responsible for obtaining any rights you may require for your implementation. Xilinx expressly disclaims any warranty whatsoever with respect to the adequacy of the implementation, including but not limited to any warranties or representations that this implementation is free from claims of infringement and any implied warranties of merchantability or fitness for a particular purpose.

# Functional Description

The floating-point and fixed-point representations employed by the core are described in "Floating-Point Number Representation" on page 2 and "Fixed-Point Number Representation" on page 4.

## Floating-Point Number Representation

The core employs a floating-point representation that is a generalization of the *IEEE-754 Standard* to allow for non-standard sizes [1]. When standard sizes are chosen, the format and special values employed are identical to those described by the *IEEE-754 Standard*.

Two parameters have been adopted for the purposes of generalizing the format processed by the Floating-Point core. These specify the total format width and the width of the fractional part. For standard single precision types, the format width is 32 bits and fraction width 24 bits. In the following description, these widths are abbreviated to $w$ and $w_f$, respectively.

A floating-point number is represented using a sign, exponent, and fraction (which are denoted as 's,' 'E,' and $b_0.b_1b_2\ldots b_{w_f-1}$, respectively).

The value of a floating-point number is given by: $v = (-1)^s 2^E b_0.b_1b_2\ldots b_{w_f-1}$

The binary bits, $b_i$, have weighting $2^{-i}$, where the most significant bit $b_0$ is a constant 1. As such, the combination is bounded such that $0 \leq b_0.b_1b_2\ldots b_{p-1} < 2$ and the number is said to be normalized. To provide increased dynamic range, this quantity is scaled by a positive or negative power of 2 (denoted here as E). The sign bit provides a value that is negative when $s = 1$, and positive when $s = 0$.

The binary representation of a floating-point number contains three fields as shown in Figure 2.

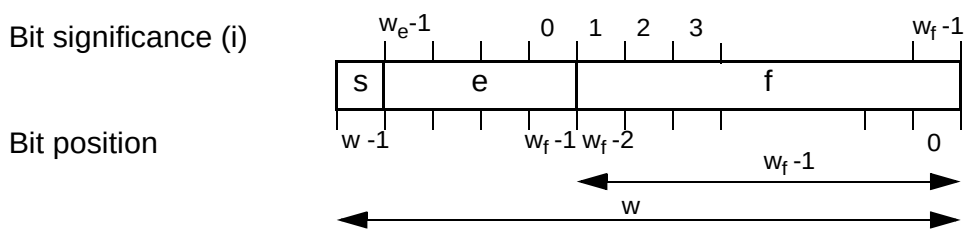


*Figure 2:* **Bit Fields Within the Floating-Point Representation**

As $b_0$ is a constant, only the fractional part is retained, that is, $f = b_1\ldots b_{w_f-1}$. This requires only $w_f - 1$ bits. Of the remaining bits, one bit is used to represent the sign, and $w_e = w - w_f$ bits represent the exponent.

The exponent field, $e$, employs a biased unsigned integer representation, whose value is given by:

$$e = \sum_{i=0}^{w_e-1} e_i 2^i$$

The index, i, of each bit within the exponent field is given in Figure 2.

The value of the exponent, $E$, is obtained by removing the bias, that is, $E = e - (2^{w_e-1} - 1)$.

In reality, $w_f$ is not the wordlength of the fraction, but the fraction with the hidden bit, $b_0$, included. This terminology has been adopted to provide commonality with that used to describe fixed-point parameters (as employed by Xilinx System Generator™ for DSP).

### Special Values

A number of values for $s$, $e$ and $f$ have been reserved for representing special numbers, such as Not a Number (NaN), Infinity (∞), zero (0), and de-normalized numbers. These special values are summarized in Table 1.

*Table 1:* **Special Values**

| Symbol for Special Value | s field | e field | f field |
|---|---|---|---|
| NaN | don't care | $2^{w_e - 1}$ (that is, $e = 11...11$) | most significant bit of fraction set (that is, $f = 10...00$) |
| $\pm\infty$ | sign of $\infty$ | $2^{w_e - 1}$ (that is, $e = 11...11$) | zero (that is, $f = 00...00$)j |
| $\pm 0$ | sign of $0$ | 0 | zero (that is, $f = 00...00$) |
| denormalized | sign of number | 0 | any non-zero field |

Note that in Table 1 the sign bit is undefined when a result is a NaN. Also, infinity and zero are signed. Where possible, the sign is handled in the same way as finite non-zero numbers. For example, $-0 + (-0) = -0$, $-0 + 0 = 0$ and $-\infty + (-\infty) = -\infty$. Whereas, a meaningless operation such as $-\infty + \infty$ will raise an invalid operation exception and produce a NaN as a result.

## IEEE-754 Support

The Xilinx Floating-Point core complies with much of the *IEEE-754 Standard*. The deviations generally provide better trade-off of resources against functionality. Specifically, the core deviates in the following ways:

- Non-Standard Wordlengths
- Denormalized Numbers
- Rounding Modes
- Signalling and Quiet NaNs

### Non-Standard Wordlengths

The Xilinx Floating-Point core supports a greater range of fraction and exponent wordlength than defined in the *IEEE-754 Standard*.

Standard formats commonly implemented by programmable processors:

- **Single Format** - uses 32 bits, with a 24-bit fraction and 8-bit exponent.
- **Double Format** - uses 64 bits, with 53-bit fraction and 11-bit exponent.

Less commonly implemented standard formats are:

- **Single Extended** - wordlength extensions of 43 bits and above
- **Double Extended** - wordlength extensions of 79 bits and above

The Xilinx core supports formats with fraction and exponent wordlengths outside of these standard wordlengths.

### Denormalized Numbers

Denormalized numbers are not supported by the Xilinx Floating-Point core. To provide robustness, the core treats denormalized numbers as zero (that takes on the sign of the denormalized number).

Denormalized numbers are those where $b_0$ is 0. As such, $b_0.b_1b_2\ldots b_{p-1} < 1$, which for a given exponent wordlength allows numbers to be represented that are smaller than otherwise possible. But note that as the value becomes smaller, it is represented with fewer bits and the relative rounding error introduced by each operation increases. An alternative way of increasing dynamic range, which uses less resources, is to increase the wordlength of the exponent. The wordlength of the format can be maintained by increasing the exponent wordlength at the expense of the fraction.

**Note:** The support for denormalized numbers cannot be switched off on some processors. Therefore, there may be very small differences between values generated by the Floating-Point core and a program running on a conventional processor when numbers are very small. If such differences must be avoided, the arithmetic model on the conventional processor should include a simple check for denormalized numbers. This check should set the output of an operation to zero when denormalized numbers are detected to correctly reflect what happens in the FPGA implementation.

### Rounding Modes

Currently, only the default rounding mode, Round to Nearest, as defined by the *IEEE-754 Standard*, is supported.

### Signalling and Quiet NaNs

The *IEEE-754 Specification* requires provision of Signalling and Quiet NaNs. However, the Xilinx Floating-Point core treats all NaNs as Quiet NaNs. When any NaN is supplied as one of the operands to the core, the result will be a Quiet NaN, and an invalid operation exception will not be raised (as would be the case for signalling NaNs). The exception to this rule is floating-point to fixed-point conversion. For detailed information, see the behavior of INVALID_OP.

## Fixed-Point Number Representation

A fixed-point representation is adopted that is consistent with the signed integer type used by Xilinx System Generator for DSP. Fixed-point values are represented using a 2s complement number weight by a fixed power of 2. The overall wordlength is specified by the format width with the level of scaling specified by the fraction width. The binary representation of a fixed-point number can be considered to contain three fields as shown in Figure 3 (although it is simply a 2s complement number).

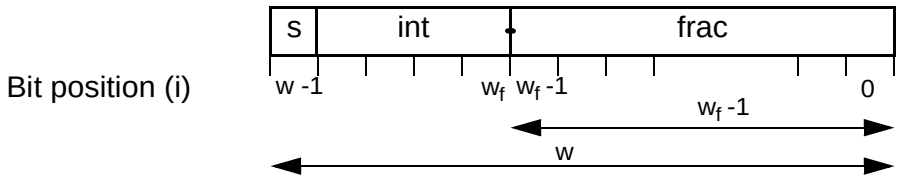


*Figure 3:* **Bit Fields within the Fixed-point Representation**

In Figure 3, the bit position has been labelled with an index i. Based upon this, the value of a fixed-point number is given by:

$$v = (-1)^{s} 2^{w-1-w_f} + b_{w-2} \ldots b_{w_f} . b_{w_f-1} \ldots b_1 b_0$$

$$= (-1)^{b_{w-1}} 2^{w-1-w_f} + \sum_{0}^{w-2} 2^{i-w_f} b_i$$

For example, a 32-bit signed integer representation is obtained when a width of 32 and a fraction width of 0 are specified. Round to Nearest is employed within the conversion operations.

**Note:** To provide for the sign bit, the width of the integer field must be at least 1, requiring that the fractional width be no larger that width-1.

## Port Description

The ports employed by the core are shown in Figure 1. They are described in more detail in Table 2. All control signals are active high.

*Table 2:* **Core Ports**

| Name | Width | Direction | Description |
|---|---|---|---|
| A[1] | w | INPUT | **Operand A** |
| B[1] | w | INPUT | **Operand B:** Only present on binary operation. |
| OPERATION[1] | 6 | INPUT | **Operation:** Specifies the operation to be performed. Implemented when the core is configured for both add and subtract operations, or as a programmable comparator. |
| OPERATION_ND | 1 | INPUT | **New Data**: Must be set high to indicate that operand A, operand B and OPERATION, when required, are valid. |
| OPERATION_RFD | 1 | OUTPUT | **Ready For Data**: Set high by core to indicate that it is ready for new operands. |
| SCLR | 1 | INPUT | **Synchronous Reset** (optional). |
| CE | 1 | INPUT | **Clock Enable** (optional). |
| CLK | 1 | INPUT | **Clock** |
| RESULT | w | OUTPUT | **Result Output**: Result of operation. |
| UNDERFLOW | 1 | OUTPUT | **Underflow:** Set high by core when underflow occurs. Supplied in synchronism with associated RESULT. |
| OVERFLOW | 1 | OUTPUT | **Overflow:** Set high by core when overflow occurs. Supplied in synchronism with associated RESULT. |

*Table 2:* **Core Ports *(Continued)***

| Name | Width | Direction | Description |
|---|---|---|---|
| INVALID_OPERATION | 1 | OUTPUT | **Invalid Operation:** Set high by core when operands cause an invalid operation. Supplied in synchronism with associated RESULT. |
| DIVIDE_BY_ZERO | 1 | OUTPUT | **Divide By Zero:** Set high by a divide operation to indicate that a division by zero was performed. Supplied in synchronism with associated RESULT. |
| RDY | 1 | OUTPUT | **Output Ready**: Set high by core when RESULT is valid. |

1. A, B and OPERATION are not registered on the input to the core. Should this be required, registers can be added to these inputs externally to the core.

### A

Operand `A` input.

### B

Operand `B` input.

### CLK

All signals are synchronous to the `CLK` input.

### CE

When `CE` is deasserted, the clock is disabled, and the state of the core and its outputs are maintained.

### SCLR

When `SCLR` is asserted, the core control is synchronously set to its initial state. Any incomplete results are discarded, and `RDY` will not be generated for them. While `SCLR` is asserted both `OPERATION_RFD` and `RDY` are deasserted. The core is ready for new input one cycle after `SCLR` is deasserted, at which point `OPERATION_RFD` is asserted.

### OPERATION

`OPERATION` is present when add and subtract operations are selected together, or when a programmable comparator is selected. The operations are binary encoded as specified in Table 3.

*Table 3:* **Encoding of OPERATION**

| FP operation | | OPERATION (5 downto 0) |
|---|---|---|
| Add | | 000000 |
| Subtract | | 000001 |
| Compare (Programmable) | Unordered | 000100 |
| | Less Than | 001100 |
| | Equal | 010100 |
| | Less Than or Equal | 011100 |
| | Greater Than | 100100 |
| | Not Equal | 101100 |
| | Greater Than or Equal | 110100 |

### OPERATION_ND

OPERATION_ND should be asserted when operands are valid on inputs A and B and the FP operation is valid on OPERATION (should it be required). Deasserting OPERATION_ND will prevent the initiation of new operations and the subsequent assertion of RDY.

**Note:** OPERATION_ND is required to synchronize operations when the core is configured to perform a multi-cycle divide or square root.

### OPERATION_RFD

OPERATION_RFD is asserted by the core to indicate that it is ready to accept new operands on inputs A, B, and OPERATION. A new operation will be initiated by the core when both OPERATION_ND and OPERATION_RFD are asserted together.

### RESULT

If the operation is compare, then the valid bits within the result depend upon the compare operation selected. If the operation is one of those listed in Table 3, then only the least significant bit of the result indicates whether the comparison is true or false. If the operation is condition code, then 4 bits provide the results of the comparison using the encoding summarized in Table 4. See *IEEE-754 Standard* for a more complete listing of the meanings of all the valid comparison results.

*Table 4:* **Condition Code Summary**

| Compare Operation | Result(3 down to 0) | | | | Result |
|---|---|---|---|---|---|
| | **3** | **2** | **1** | **0** | |
| Programmable | | | | 0 | A OP B = False |
| | | | | 1 | A OP B = True |
| Condition Code | Unordered | > | < | EQ | Meaning |
| | 0 | 0 | 0 | 1 | A = B |
| | 0 | 0 | 1 | 0 | A < B |
| | 0 | 0 | 1 | 1 | A <= B |
| | 0 | 1 | 0 | 0 | A > B |
| | 0 | 1 | 0 | 1 | A >= B |
| | 0 | 1 | 1 | 0 | A <> B |
| | 1 | See Standard | | | A, B or both are NaN. |

The following signals provide exception information. Additional detail on their behavior can be found in the *IEEE-754 Standard*.

### UNDERFLOW

Underflow is signalled when the operation generates a non-zero result which is too small to be represented with the chosen precision. The result is set to zero. Underflow is detected after rounding.

**Note:** A number that becomes de-normalized before rounding will be set to zero and underflow signalled.

### OVERFLOW

Overflow is signalled when the operation generates a result that is too large to be represented with the chosen precision. The output is set to a correctly signed $\infty$.

### INVALID_OP

Invalid operation is signalled when the operation performed is invalid. According to the *IEEE-754 Standard*, the following are invalid operations:

1. Any operation on a signalling NaN. (Note that this is not relevant here).
2. Addition or subtraction of infinite values where the sign of the result cannot be determined. For example, magnitude subtraction of infinities such as $(+\infty) + (-\infty)$.
3. Multiplication where $0 \times \infty$.
4. Division where $0/0$ or $\infty/\infty$.
5. Square root if the operand is less than zero.
6. When the input of a conversion cannot correctly signalled by the result (for example NaN or infinity).

When an invalid operation occurs, the associated result is a Quiet NaN. In the case of floating-point to fixed-point conversion, NaN and infinity raise an invalid operation exception. If the operand is out of range, or an infinity, then an overflow exception is raised. By analyzing the two exception signals it is possible to determine which of the three types of operand were converted. (See Table 5.)

*Table 5:* **Invalid Operation Summary**

| Operand | Invalid Operation | Overflow | Result |
|---|---|---|---|
| + Out of Range | 0 | 1 | 011...11 |
| - Out of Range | 0 | 1 | 100...00 |
| + Infinity | 1 | 1 | 011...11 |
| - Infinity | 1 | 1 | 100...00 |
| NaN | 1 | 0 | 100...00 |

When the operand is a NaN the result is set to the most negative representable number. When the operand is infinity or an out-of-range floating-point number, the result is saturated to the most positive or most negative number, depending upon the sign of the operand.

**Note:** Floating-point to fixed-point conversion does not treat a NaN as a Quiet NaN, because NaN is not representable within the resulting fixed-point format, and so can only be indicated through an invalid operation exception.

### DIVIDE_BY_ZERO

Division of a number by zero is signalled when a divide operation is performed where the divisor is zero and the dividend is a finite non-zero number. The result in this circumstance is a correctly signed $\infty$.

### RDY

`RDY` is asserted by the core to indicate that `RESULT` is valid. `RDY` can be used to qualify the result of a multi-cycle operation (i.e., divide or square root operations with rate greater than 1).

### Example Timing

An example of signal timing is given in Figure 4 for square-root with latency 4 and rate 3. The result is provided four cycles after an active OPERATION_ND. In this example, new inputs are applied every three cycles, in accordance with the maximum rate and OPERATION_RFD output. (Data could be applied less frequently, in which case, OPERATION_RFD would stay High until OPERATION_ND was asserted with the new input.) The RDY output indicates when RESULT, and any exception flags, are valid. In this example, an overflow exception has been generated with result R2.

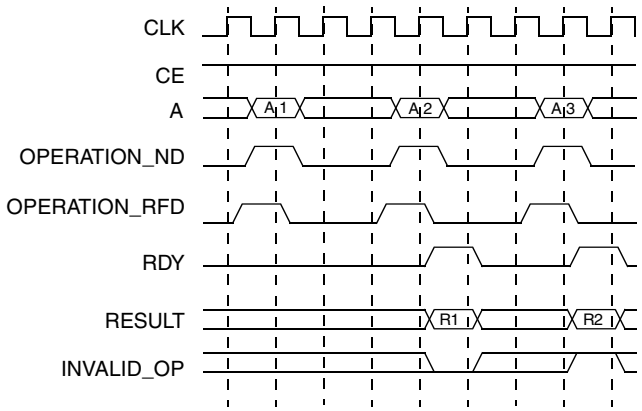


*Figure 4:* **Example Timing Diagram**

## Customizing the Core

Floating-Point core customization options are provided to optimize the core for specific requirements. The core can be customized in either of two ways:

- Using the CORE Generator Graphical User Interface (GUI)
- Instancing the core with appropriate generic values in VHDL and synthesizing the core using XST

### Using the CORE Generator GUI

The Floating-Point core GUI can be used to configure:

- Core performance
- Implementation optimizations, including wordlength
- Core operation
- Inclusion or exclusion of optional pins

#### Floating-Point Customization Options

From the CORE Generator, the Floating-Point core is located in the Math Function category. Up to five customization screens are provided, depending on the options you select. All five screens share common navigation options:

- **Next and Back**: Moves forward or backward one screen at a time, respectively.
- **Finish:** Generates the core with the currently configured parameters.

- **Cancel**: Cancels generation of the core and returns to the first screen of the GUI.
- **View Data Sheet**: Displays this document, a PDF file of the core product specification.

### Opening the Floating-Point Main Screen

1. Start the CORE Generator.
2. Select Floating-Point 3.0 from the Math Functions category at the left side of the application window.
3. Do one of the following to display the Floating-Point main configuration screen:
   - Double-click Floating-Point 3.0 in the Math Functions category.
   - Select Floating-Point in the Math Functions category; then click Customize at the right side of the CORE Generator window.

### Main Configuration Screen

The main configuration screen allows the following parameters to be specified:

- Component Name
- Operation Type

#### Component Name

The component name is used as the base name of the output files generated for the core. Names must start with a letter and be composed using the following characters: a to z, 0 to 9, and "_".

#### Operation Type

The floating-point operation may be one of the following:

- **Add/Subtract**
- **Multiply**
- **Divide**
- **Square-root**
- **Compare**
- **Fixed-to-float**
- **Float-to-fixed**
- **Float-to-float**

When *Add/Subtract* is selected, it is possible for the core to perform both operations, or just add or subtract. When both are selected, the operation performed on a particular set of operands is controlled by the `OPERATION` input (with encoding defined earlier in Table 3).

When *Add/Subtract or Multiply* is selected, the level of embedded multiplier usage can be specified as described in the Penultimate Configuration Screen section.

When *Compare* is selected, the compare operation may be programmable or fixed. If programmable, then the compare operation performed should be supplied via the `OPERATION` input (with encoding defined earlier in Table 3). If a fixed operation is required, then the operation type should be selected.

When *Float-to-float* conversion is selected, and exponent and fraction widths of the input and result are the same, the core provides a means to condition numbers, i.e., convert denormalized numbers to zero, and signal NaNs to quiet NaNs.

## Second and Third Configuration Screens

Depending on the configuration you select from the first screen, the second and third configuration screens let you specify the precision of the operand and result.

### Precision of the Operand and Results

This parameter defines the number of bits used to represent quantities. The type of the operands and results depend on the operation requested. For fixed-point conversion operations, either the operand or result is fixed-point. For all other operations, the output is specified as a floating-point type.

**Note:** For compare, depending upon operation selected, RESULT(3 down to 0), is used to indicate the result of the comparison operation.

Table 6 defines the general limits of the format widths.

*Table 6:* **General Limits of Width and Fraction Width**

| Format Type | Fraction Width | | Exponent/Integer Width (Width-Fraction Width) | | Width | |
|---|---|---|---|---|---|---|
| | Min | Max | Min | Max | Min | Max |
| Floating-Point | 4 | 64 | 4 | 16 | 4 | 64 |
| Fixed-Point | 0 | 63 | 1 | 64 | 4 | 64 |

There are also a number of further limits for specific cases:

- The exponent width (i.e., width - fraction width) should be chosen to support normalization of the fractional part. This can be calculated using:

  $$\text{exponent width} = \text{ceil}[\log_2(\text{fraction width}+3)] + 1$$

  For example, a 24-bit fractional part requires an exponent of at least 6 bits (for example, {ceil [$\log_2$ (27)]+1}). The GUI enforces these limits.
- For the logic assisted multiplier (that is, when multiplier usage is medium), only double precision format is supported.
- For conversion operations, the exponent width of the floating-point input or output can be calculated using:

  $$\text{exponent width} = \text{ceil}[\log_2(\text{width} + 3)] + 1$$

  For example, a 32-bit integer will require a minimum exponent of 7 bits.

  A summary of the width limits imposed by exponent width is provided in Table 7.

*Table 7:* **Summary of Exponent Width Limits**

| Exponent Width | Floating-Point Fraction Width or Fixed-Point Total Width |
|---|---|
| 4 | 4 to 5 |
| 5 | 6 to 13 |
| 6 | 14 to 29 |
| 7 | 30 to 61 |
| 8 | 61 to 64 |

### Penultimate Configuration Screen

The final configuration screen lets you specify the following:

- **Architecture Optimizations**
- **Family Optimizations**
- **Cycles Per Operation (Rate)**

#### Architecture Optimizations

For addition/subtraction on Virtex-5 FPGAs, it is possible to specify a latency optimized architecture, or speed optimized architecture. The latency optimized architecture offers reduced latency at the expense of increased resources.

#### Family Optimizations

- **Multiplier Usage:** Allows the type and level of embedded multiplier usage to be specified.

#### Multiplier Usage

The level of embedded multiplier usage can be specified. The level and type of multiplier usage also depend upon the operation and FPGA family. Table 8 summarizes these options for multiplication.

*Table 8:* **Impact of Family and Multiplier Usage on the Implementation of the Multiplier**

| Multiplier Usage | Virtex-II, Virtex-II Pro, Spartan-3E | Virtex-4 | Virtex-5 |
|---|---|---|---|
| No usage | Logic | Logic | Logic |
| Medium usage | Not supported | DSP48+logic[1] in multiplier body | DSP48E+logi[1] in multiplier body |
| Full usage | MULT18X18 | DSP48 used in multiplier body | DSP48E used in multiplier body |
| Max usage | Not supported | DSP48 multiplier body and rounder | DSP48E multiplier body and rounder |

1. Logic-assisted multiplier variant is only available for single and double precision in Virtex-4 FPGAs and single precision in Virtex-5 FPGAs.

Table 9 summarizes these options for addition/subtraction.

*Table 9:* **Impact of Family, Precision, and Multiplier Usage on the Implementation of the Adder/Subtractor**

| Multiplier Usage (only valid values listed) | Virtex-II, Virtex-II Pro, Spartan-3E | Virtex-4 | | | Virtex-5 | | |
|---|---|---|---|---|---|---|---|
| | Any | Other | Single | Double | Other | Single | Double |
| No usage | Logic | Logic | Logic | Logic | Logic | Logic | Logic |
| Full usage | Not supported | Not supported | 4 DSP48 | 3 DSP48 | Not supported | 2 DSP48E | 3 DSP48E |

### Latency

This parameter describes the number of cycles between an operand input and result output. The latency of all operators (apart from the logic-assisted, double-precision multipliers on Virtex-4 devices) can be set between 0 and a maximum value that is dependent upon the parameters chosen. The maximum latency of the Floating-Point core is tabulated for a range of width and operation types in Table 10, Table 11, Table 12, Table 13, Table 14, Table 15, Table 16, Table 17, Table 18, and Table 19.

The maximum latency of the divide and square root operations is fraction width + 4, and for compare operation it is three cycles. The float-to-float conversion operation is three cycles when either mantissa or exponent width is being reduced, otherwise it is two cycles. Note that it is two cycles, even when the input and result widths are the same, as the core provides conditioning in this situation (see Operation Type for further details).

**Note:** The maximum latency of certain operations has been increased over Floating Point Operator v2.0 to increase maximum clock frequency. If the previous maximum latency value is specified, then an equivalent implementation will be obtained.

*Table 10:* **Latency of Floating-Point Multiplication using Logic Only**

| Fraction Width | Maximum Latency (clock cycles) |
|---|---|
| 4 to 5 | 5 |
| 6 to 11 | 6 |
| 12 to 23 | 7 |
| 24 to 47 (single) | 8 |
| 48 to 64 (double) | 9 |

*Table 11:* **Latency of Floating-Point Multiplication using MULT18X18S**

| Fraction Width | Maximum Latency (clock cycles) |
|---|---|
| 4 to 17 | 4 |
| 18 to 34 (single) | 6 |
| 35 to 51 | 7 |
| 52 to 64 (double) | 8 |

*Table 12:* **Latency of Floating-Point Multiplication using DSP48**

| Fraction Width | Maximum Latency (clock cycles) | | |
|---|---|---|---|
| | Medium Usage | Full Usage | Max Usage |
| 4 to 17 | | 6 | 8 |
| 18 to 34 (single) | 9[1] | 10 | 11 |
| 35 to 51 | | 15 | 16 |
| 52 to 64 (double) | 17[2] | 22 | 23 |

1. Single precision only
2. Double precision only

*Table 13:* **Latency of Floating-Point Multiplication using DSP48E**

| Fraction Width | Maximum Latency (clock cycles) | | |
|---|---|---|---|
| | Medium Usage | Full Usage | Max Usage |
| 4 to 17 | | 6 | 8 |
| 18 to 24 (single) | 8[1] | 8 | 9 |
| 25 to 34 | | 10 | 11 |
| 35 to 41 | | 12 | 13 |
| 42 to 51 | | 15 | 16 |
| 52 to 58 (double) | | 18 | 19 |
| 59 to 64 | | 22 | 23 |

1. Single precision only

*Table 14:* **Latency of Floating-Point Addition using Medium Usage and DSP48/DSP48E**

| Fraction Width | Maximum Latency (clock cycles) | |
|---|---|---|
| | DSP48 | DSP48E |
| 24 (single) | 16 | 16 |
| 53 (double) | 12 | 15 |

*Table 15:* **Latency of Floating-Point Addition using Logic on Families Other than Virtex-5 FPGAs**

| Fraction Width | Maximum Latency (clock cycles) |
|---|---|
| | Virtex-II, Virtex-II Pro, Spartan-3E, Virtex-4 |
| 4, 5 | 9 |
| 6 to 14 | 10 |
| 15 | 11 |
| 16, 17 | 12 |
| 18 to 29 | 13 |
| 30 to 62 | 14 |
| 63, 64 | 15 |

*Table 16:* **Latency of Floating-Point Addition using Logic and Low-Latency Optimization on Virtex-5 FPGAs**

| Fraction Width | Maximum Latency (clock cycles) |
|---|---|
| | Virtex-5 |
| single | 9 |
| double | 9 |

*Table 17:* **Latency of Floating-Point Addition using Logic and Speed Optimization on Virtex-5 FPGAs**

| Fraction Width | Maximum Latency (clock cycles) |
|---|---|
| | Virtex-5 |
| 4 to 13 | 8 |
| 14 | 9 |
| 15 | 10 |
| 16, 17 | 11 |
| 18 to 61 (single, double) | 12 |
| 62 to 64 | 13 |

*Table 18:* **Latency of Fixed-point to Floating-Point Conversion**

| Operand Width | Maximum Latency (Cycles) |
|---|---|
| 4 to 8 | 5 |
| 9 to 32 | 6 |
| 31 to 64 | 7 |

*Table 19:* **Latency of Floating-Point to Fixed-point Conversion**

| Maximum of (A Fraction Width+1) and Result Width | Maximum Latency (Cycles) |
|---|---|
| 4 | 4 |
| 5 to 16 | 5 |
| 17 to 64 | 6 |

### Cycles Per Operation (Rate)

This parameter describes the minimum number of cycles that must elapse between inputs. This rate can be specified. A value of 1 allows operands to be applied on every clock cycle, and results in a fully-parallel circuit. A value greater than 1 enables hardware reuse. The number of slices consumed by the core reduces as the number of cycles per operation is increased. A value of 2 approximately halves the number of slices used. A fully sequential implementation is obtained when the value is equal to fraction width+1 for the square-root operation, and fraction width+2 for the divide operation.

## Final Configuration Screen

The final configuration screen lets you specify the Optional Control and Exception Pins.

### Optional Control and Exception Pins

Pins for the following signals are optional:

- **Control Signals:** `OPERATION_ND`, `OPERATION_RDY`, `RDY`, `CE` and `SCLR` control signals are optional.
- **Exception Signals:** `UNDERFLOW`, `OVERFLOW`, `INVALID_OPERATION` and `DIVIDE_BY_ZERO` signals are optional. The `DIVIDE_BY_ZERO` signal is only available when the divide operation is selected.

## VHDL Interface

The Floating-Point core can be generated directly from a component instantiation in VHDL using XST. A component declaration has been provided in `xilinxcorelib` (as employed by XST). Also, a package of constant definitions has been provided to allow parameter values to be used that have meaningful names.

**Note:** If the generics are out of range, then the core will fail to synthesize. If this happens, either simulate the VHDL behavioral model (see VHDL Simulation) to obtain the reason for failure, or use the GUI to identify a suitable set of generics.

A list of valid generics and their limits and default values is listed in Table 20.

*Table 20:* **Parameter File Information**

| VHDL Generic | Valid Values | Default Value |
|---|---|---|
| C_FAMILY | virtex2, virtex2p, virtex4, virtex5, spartan3 Use one of these families for derivatives (such as spartan3e). | virtex2 |
| C_HAS_ADD | FLT_PT_TRUE, FLT_PT_FALSE Can be used with C_HAS_SUBTRACT to obtain add and subtract capability. When both are selected the OPERATION port is used to specify required operation as defined in Table 3. | FLT_PT_FALSE |
| C_HAS_SUBTRACT | FLT_PT_TRUE, FLT_PT_FALSE Can be used with C_HAS_ADD to obtain add and subtract capability. (See further comments under C_HAS_ADD). | FLT_PT_FALSE |
| C_HAS_MULTIPLY | FLT_PT_TRUE, FLT_PT_FALSE | FLT_PT_FALSE |
| C_HAS_DIVIDE | FLT_PT_TRUE, FLT_PT_FALSE | FLT_PT_FALSE |
| C_HAS_SQRT | FLT_PT_TRUE, FLT_PT_FALSE | FLT_PT_FALSE |
| C_HAS_COMPARE | FLT_PT_TRUE, FLT_PT_FALSE | FLT_PT_FALSE |
| C_HAS_FIX_TO_FLT | FLT_PT_TRUE, FLT_PT_FALSE | FLT_PT_FALSE |
| C_HAS_FLT_TO_FIX | FLT_PT_TRUE, FLT_PT_FALSE | FLT_PT_FALSE |
| C_HAS_FLT_TO_FLT | FLT_PT_TRUE, FLT_PT_FALSE | FLT_PT_FALSE |
| C_A_WIDTH | Integer with range dependant upon other parameters as defined in Precision of the Operand and Results. | 32 |
| C_A_FRACTION_WIDTH | Integer with range dependant upon other parameters as defined in Precision of the Operand and Results. | 24 |
| C_B_WIDTH | Must be same as C_A_WIDTH. | 32 |
| C_B_FRACTION_WIDTH | Must be same as C_A_FRACTION_WIDTH. | 24 |
| C_RESULT_WIDTH | Must be same as C_A_WIDTH for all operations, other than conversion. | 32 |
| C_RESULT_FRACTION_WIDTH | Must be same as C_A_FRACTION_WIDTH for all operations, other than conversion. | 24 |

*Table 20:* **Parameter File Information *(Continued)***

| VHDL Generic | Valid Values | Default Value |
|---|---|---|
| C_COMPARE_OPERATION | FLT_PT_PROGRAMMABLE, FLT_PT_LESS_THAN, FLT_PT_LESS_THAN, FLT_PT_EQUAL, FLT_PT_LESS_THAN_OR_EQUAL, FLT_PT_GREATER_THAN, FLT_PT_NOT_EQUAL, FLT_PT_GREATER_THAN_OR_EQUAL , FLT_PT_UNORDERED, FLT_PT_CONDITION_CODE. Specify when operation is compare. | FLT_PT_LESS_THAN |
| C_LATENCY | Integer with range dependant upon other parameters as defined in Final Configuration Screen. | FLT_PT_MAX_LATENCY |
| C_OPTIMIZATION | FLT_PT_SPEED_OPTIMIZED, FLT_PT_LOW_LATENCY. | FLT_PT_SPEED_OPTIMIZED |
| C_MULT_USAGE | FLT_PT_NO_USAGE, FLT_PT_MEDIUM, FLT_PT_FULL_USAGE, FLT_PT_MAX_USAGE. Use as described in Multiplier Usage. | FLT_PT_FULL_USAGE |
| C_HAS_CE | FLT_PT_TRUE, FLT_PT_FALSE | FLT_PT_FALSE |
| C_HAS_SCLR | FLT_PT_TRUE, FLT_PT_FALSE | FLT_PT_FALSE |
| C_HAS_OPERATION_ND | FLT_PT_TRUE, FLT_PT_FALSE | FLT_PT_FALSE |
| C_HAS_OPERATION_RFD | FLT_PT_TRUE, FLT_PT_FALSE | FLT_PT_FALSE |
| C_HAS_RDY | FLT_PT_TRUE, FLT_PT_FALSE | FLT_PT_FALSE |
| C_HAS_UNDERFLOW | FLT_PT_TRUE, FLT_PT_FALSE | FLT_PT_FALSE |
| C_HAS_OVERFLOW | FLT_PT_TRUE, FLT_PT_FALSE | FLT_PT_FALSE |
| C_HAS_INVALID_OPERATION | FLT_PT_TRUE, FLT_PT_FALSE | FLT_PT_FALSE |
| C_HAS_DIVIDE_BY_ZERO | FLT_PT_TRUE, FLT_PT_FALSE | FLT_PT_FALSE |

## Examples

The following is an example of an instantiation in VHDL that generates a single-precision adder with a full set of exception and control signals.

```
library xilinxcorelib; -- XST version
use xilinxcorelib.floating_point_v3_0_consts.all; -- constants package
use xilinxcorelib.floating_point_v3_0_comp.all; -- component declaration
.....
fp_add_single: floating_point_v3_0
    generic map (
   C_FAMILY                 => "virtex4",
   C_HAS_ADD                => FLT_PT_TRUE,
   C_A_WIDTH                => 32,
   C_A_FRACTION_WIDTH       => 24,
   C_B_WIDTH                => 32,
   C_B_FRACTION_WIDTH       => 24,
```

```
    C_RESULT_WIDTH           => 32,
    C_RESULT_FRACTION_WIDTH => 24,
    C_HAS_SCLR               => FLT_PT_TRUE,
    C_HAS_OPERATION_ND       => FLT_PT_TRUE,
    C_HAS_OPERATION_RFD      => FLT_PT_TRUE,
    C_HAS_RDY                => FLT_PT_TRUE,
    C_HAS_UNDERFLOW          => FLT_PT_TRUE,
    C_HAS_OVERFLOW           => FLT_PT_TRUE,
    C_HAS_INVALID_OP         => FLT_PT_TRUE
)
port (
    A                        => a,
    B                        => b,
    OPERATION_ND             => operation_nd,
    OPERATION_RFD            => operation_rfd,
    CLK                      => clk,
    SCLR                     => sclr,
    RESULT                   => result,
    UNDERFLOW                => underflow,
    OVERFLOW                 => overflow,
    INVALID_OP               => invalid_op,
    RDY                      => rdy
);
```

### VHDL Component Declaration

This component declaration is provided within `xilinxcorelibs` in package floating_point_v3_0_comp. It is included below for reference purposes. Note that there are a number of generics over-and-above those defined in Table 20. The values of these additional generics should not be changed from the defaults, and can be left unspecified when the component is instantiated (as done in the example). There are also a number of ports in addition to those listed in Table 2. These should be left unconnected. Inputs will default to suitable values.

```
component floating_point_v3_0 is
    generic (
    C_FAMILY               : string  := C_FAMILY_DEFAULT;
    C_HAS_ADD              : integer := C_HAS_ADD_DEFAULT;
    C_HAS_MULTIPLY         : integer := C_HAS_MULTIPLY_DEFAULT;
    C_HAS_DIVIDE           : integer := C_HAS_DIVIDE_DEFAULT;
    C_HAS_SQRT             : integer := C_HAS_SQRT_DEFAULT;
    C_HAS_COMPARE          : integer := C_HAS_COMPARE_DEFAULT;
    C_HAS_FIX_TO_FLT       : integer := C_HAS_FIX_TO_FLT_DEFAULT;
    C_HAS_FLT_TO_FIX       : integer := C_HAS_FLT_TO_FIX_DEFAULT;
    C_HAS_FLT_TO_FLT       : integer := C_HAS_FLT_TO_FLT_DEFAULT;
    C_A_WIDTH              : integer := C_A_WIDTH_DEFAULT;
    C_A_FRACTION_WIDTH     : integer := C_A_FRACTION_WIDTH_DEFAULT;
    C_B_WIDTH              : integer := C_B_WIDTH_DEFAULT;
    C_B_FRACTION_WIDTH     : integer := C_B_FRACTION_WIDTH_DEFAULT;
    C_RESULT_WIDTH         : integer := C_RESULT_WIDTH_DEFAULT;
    C_RESULT_FRACTION_WIDTH: integer := C_RESULT_FRACTION_WIDTH_DEFAULT;
    C_COMPARE_OPERATION    : integer := C_COMPARE_OPERATION_DEFAULT;
    C_LATENCY              : integer := C_LATENCY_DEFAULT;
    C_OPTIMIZATION         : integer := C_OPTIMIZATION_DEFAULT;
    C_MULT_USAGE           : integer := C_MULT_USAGE_DEFAULT;
```

```
    C_RATE                  : integer := C_RATE_DEFAULT;
    C_HAS_ACLR              : integer := C_HAS_ACLR_DEFAULT;
    C_HAS_CE                : integer := C_HAS_CE_DEFAULT;
    C_HAS_SCLR              : integer := C_HAS_SCLR_DEFAULT;
    C_HAS_A_NEGATE          : integer := C_HAS_A_NEGATE_DEFAULT;
    C_HAS_B_NEGATE          : integer := C_HAS_B_NEGATE_DEFAULT;
    C_HAS_A_ND              : integer := C_HAS_A_ND_DEFAULT;
    C_HAS_A_RFD             : integer := C_HAS_A_RFD_DEFAULT;
    C_HAS_B_ND              : integer := C_HAS_B_ND_DEFAULT;
    C_HAS_B_RFD             : integer := C_HAS_B_RFD_DEFAULT;
    C_HAS_OPERATION_ND      : integer := C_HAS_OPERATION_ND_DEFAULT;
    C_HAS_OPERATION_RFD     : integer := C_HAS_OPERATION_RFD_DEFAULT;
    C_HAS_RDY               : integer := C_HAS_RDY_DEFAULT;
    C_HAS_CTS               : integer := C_HAS_CTS_DEFAULT;
    C_HAS_UNDERFLOW         : integer := C_HAS_UNDERFLOW_DEFAULT;
    C_HAS_OVERFLOW          : integer := C_HAS_OVERFLOW_DEFAULT;
    C_HAS_INVALID_OP        : integer := C_HAS_INVALID_OP_DEFAULT;
    C_HAS_INEXACT           : integer := C_HAS_INEXACT_DEFAULT;
    C_HAS_DIVIDE_BY_ZERO    : integer := C_HAS_DIVIDE_BY_ZERO_DEFAULT;
    C_HAS_STATUS            : integer := C_HAS_STATUS_DEFAULT
    C_HAS_EXCEPTION         : integer := C_HAS_EXCEPTION_DEFAULT
    C_STATUS_EARLY          : integer := C_STATUS_EARLY_DEFAULT
);
port (
    A                       : in std_logic_vector(C_A_WIDTH-1 downto 0);
    B                       : in std_logic_vector(C_B_WIDTH-1 downto 0):=(others=>'0');
    A_NEGATE                : in std_logic:='0';
    B_NEGATE                : in std_logic:='0';
    OPERATION               : in std_logic_vector(5 downto 0):=(others=>'0');
    A_ND                    : in std_logic:='1';
    A_RFD                   : out std_logic;
    B_ND                    : in std_logic:='1';
    B_RFD                   : out std_logic;
    OPERATION_ND            : in std_logic:='1';
    OPERATION_RFD           : out std_logic;
    CLK                     : in std_logic;
    SCLR                    : in std_logic:='0';
    ACLR                    : in std_logic:='0';
    CE                      : in std_logic:='0';
    RESULT                  : out std_logic_vector(C_RESULT_WIDTH-1 downto 0);
    STATUS                  : out std_logic_vector(2 downto 0);
    EXCEPTION               : out std_logic;
    UNDERFLOW               : out std_logic;
    OVERFLOW                : out std_logic;
    INVALID_OP              : out std_logic;
    INEXACT                 : out std_logic;
    DIVIDE_BY_ZERO          : out std_logic;
    RDY                     : out std_logic;
    CTS                     : in std_logic:='1'
);
end component;
```

### VHDL Constants Package

Default generic values and constant terms for valid values of the generics are provided within `xilinx-corelibs` as the package floating_point_v3_0_consts. Some useful constants and the default generics are as follows:

```
constant FLT_PT_TRUE                    : integer := 1;
constant FLT_PT_FALSE                   : integer := 0;
constant FLT_PT_SPEED_OPTIMIZED         : integer := 1;

constant FLT_PT_NO_USAGE                : integer := 0;
constant FLT_PT_MEDIUM_USAGE            : integer := 1;
constant FLT_PT_FULL_USAGE              : integer := 2;
constant FLT_PT_MAX_USAGE               : integer := 3;
constant FLT_PT_MAX_LATENCY             : integer := 1000;

-- Compare operation values
constant FLT_PT_UNORDERED               : integer := 0;
constant FLT_PT_LESS_THAN               : integer := 1;
constant FLT_PT_EQUAL                   : integer := 2;
constant FLT_PT_LESS_THAN_OR_EQUAL      : integer := 3;
constant FLT_PT_GREATER_THAN            : integer := 4;
constant FLT_PT_NOT_EQUAL               : integer := 5;
constant FLT_PT_GREATER_THAN_OR_EQUAL   : integer := 6;
constant FLT_PT_CONDITION_CODE          : integer := 7;
constant FLT_PT_PROGRAMMABLE            : integer := 8;

constant FLT_PT_OPERATION_WIDTH         : integer := 6;

-- defaults for generics
constant C_FAMILY_DEFAULT               : string := "virtex2";
constant C_HAS_ADD_DEFAULT              : integer := FLT_PT_FALSE;
constant C_HAS_MULTIPLY_DEFAULT         : integer := FLT_PT_FALSE;
constant C_HAS_DIVIDE_DEFAULT           : integer := FLT_PT_FALSE;
constant C_HAS_SQRT_DEFAULT             : integer := FLT_PT_FALSE;
constant C_HAS_COMPARE_DEFAULT          : integer := FLT_PT_FALSE;
constant C_HAS_FIX_TO_FLT_DEFAULT       : integer := FLT_PT_FALSE;
constant C_HAS_FLT_TO_FIX_DEFAULT       : integer := FLT_PT_FALSE;
constant C_A_WIDTH_DEFAULT              : integer := 32;
constant C_A_FRACTION_WIDTH_DEFAULT     : integer := 24;
constant C_B_WIDTH_DEFAULT              : integer := 32;
constant C_B_FRACTION_WIDTH_DEFAULT     : integer := 24;
constant C_RESULT_WIDTH_DEFAULT         : integer := 32;
constant C_RESULT_FRACTION_WIDTH_DEFAULT: integer := 24;
constant C_COMPARE_OPERATION            : integer := FLT_PT_LESS_THAN;
constant C_LATENCY_DEFAULT              : integer := FLT_PT_MAX_LATENCY;
constant C_OPTIMIZATION_DEFAULT         : integer := FLT_PT_SPEED_OPTIMIZED;
constant C_MULT_USAGE_DEFAULT           : integer := FLT_PT_FULL_USAGE;
constant C_RATE_DEFAULT                 : integer := 1;
constant C_HAS_ACLR_DEFAULT             : integer := FLT_PT_FALSE;
constant C_HAS_CE_DEFAULT               : integer := FLT_PT_FALSE;
constant C_HAS_SCLR_DEFAULT             : integer := FLT_PT_FALSE;
constant C_HAS_A_NEGATE_DEFAULT         : integer := FLT_PT_FALSE;
```

```
constant C_HAS_B_NEGATE_DEFAULT        : integer := FLT_PT_FALSE;
constant C_HAS_A_ND_DEFAULT            : integer := FLT_PT_FALSE;
constant C_HAS_A_RFD_DEFAULT           : integer := FLT_PT_FALSE;
constant C_HAS_B_ND_DEFAULT            : integer := FLT_PT_FALSE;
constant C_HAS_B_RFD_DEFAULT           : integer := FLT_PT_FALSE;
constant C_HAS_OPERATION_ND_DEFAULT    : integer := FLT_PT_FALSE;
constant C_HAS_OPERATION_RFD_DEFAULT   : integer := FLT_PT_FALSE;
constant C_HAS_RDY_DEFAULT             : integer := FLT_PT_FALSE;
constant C_HAS_CTS_DEFAULT             : integer := FLT_PT_FALSE;
constant C_HAS_UNDERFLOW_DEFAULT       : integer := FLT_PT_FALSE;
constant C_HAS_OVERFLOW_DEFAULT        : integer := FLT_PT_FALSE;
constant C_HAS_INVALID_OP_DEFAULT      : integer := FLT_PT_FALSE;
constant C_HAS_INEXACT_DEFAULT         : integer := FLT_PT_FALSE;
constant C_HAS_DIVIDE_BY_ZERO_DEFAULT  : integer := FLT_PT_FALSE;
constant C_HAS_STATUS_DEFAULT          : integer := FLT_PT_FALSE;
```

# Simulation

## VHDL Simulation

A cycle-accurate, bit-true VHDL simulation model exists for the Xilinx Floating-Point core within `xilinxcorelib` library. For multi-cycle divide or square-root, in which case RATE>1, the model `RESULT` and exception flags may differ from the core in between valid outputs. `RDY` indicates when the `RESULT` and exception flags are valid and can be used to qualify these outputs from the model. Also note that the sign of a NaN is undefined, and the model and core may differ in this respect.

The `xilinxcorelib` library can be compiled using COMPXLIB for your particular simulator. See ISE documentation for further details. If the core has been directly instantiated within VHDL, then the `xilinxcorelib` library will already have been referenced in the code, and the availability of the `xilinxcorelib` library to the simulator will result in the behavioral model being used to simulate the core.

**Note:** When direct instantiation is used, XST will employ its own version of `xilinxcorelib`. The XST library is made available to XST when the IP download is installed.

Alternatively, a simulation wrapper file can be generated by CORE Generator. Within the wrapper file, the generics on the component instance are set to the same values used to generate the core. For further details on how to generate a simulation wrapper file, see the VHDL Design Flow within the CORE Generator documentation.

## Verilog Simulation

A Verilog model of the Xilinx Floating-Point core is not supplied within the Verilog version of `xilinxcorelib`. However, a Verilog structural simulation model for a specific core can be generated by CORE Generator. See the *Generation Panel under Project Options* for the controls to enable this. For further details, see *Verilog Design Flow* within the CORE Generator documentation.

## Resource Utilization and Performance

The resource requirements and maximum clock rates achievable on Spartan-3E, Virtex-4, and Virtex-5 FPGAs are summarized as follows for the case of maximum latency.

**Note:** LUT and FF resource usage and maximum frequency will reduce with latency. Minimizing latency will minimize resources.

### Custom Format: 17-Bit Fraction and 24-Bit Total Wordlength

The resource requirements and maximum clock rates achievable with 17-bit fraction and 24-bit total wordlength on Spartan-3E FPGAs are summarized in Table 21, on Virtex-4 in Table 22, and on Virtex-5 in Table 23.

*Table 21:* **Characterization of 17-Bit Fraction and 24-Bit Total Wordlength on Spartan-3E FPGA**

| Operation | Resources | | | | Maximum Frequency (MHz)[1] |
|---|---|---|---|---|---|
| | Embedded | | Fabric | | Spartan-3E |
| | Type | Number | LUTs | FFs | -4 |
| Multiply | MULT18X18S (full usage) | 1 | 92 | 110 | 167 |
| | Logic (no usage) | 0 | 339 | 401 | 172 |
| Add/Subtract | Logic (no usage) | | 438 | 430 | 195 |
| Fixed to float | Int24 input | | 166 | 176 | 214 |
| Float to fixed | Int24 result | | 195 | 190 | 217 |
| Float to float | Single to 24-17 format | | 66 | 89 | 183 |
| | 24-17 to single | | 13 | 52 | 252 |
| Compare | Programmable | | 82 | 24 | 223 |
| Divide | C_RATE=1 | | 460 | 751 | 203 |
| | C_RATE=19 | | 172 | 186 | 173 |
| Sqrt | C_RATE=1 | | 308 | 447 | 209 |
| | C_RATE=18 | | 160 | 161 | 186 |

1. Maximum frequency obtained with map switches -ol high and -cm speed, and par switches -pl high and -rl high.

*Table 22:* **Characterization of 17-Bit Fraction and 24-Bit Total Wordlength on Virtex-4 FPGA**

| Operation | Resources | | | | Maximum Frequency (MHz)[1] |
|---|---|---|---|---|---|
| | Embedded | | Fabric | | Virtex-4 |
| | Type | Number | LUTs | FFs | -10 |
| Multiply | DSP48 (max usage) | 2 | 89 | 133 | 396 |
| | DSP48 (full usage) | 1 | 103 | 129 | 354 |
| | Logic (no usage) | 0 | 345 | 401 | 267 |

*Table 22:* **Characterization of 17-Bit Fraction and 24-Bit Total Wordlength on Virtex-4 FPGA *(Continued)***

| Operation | Resources | | | | Maximum Frequency (MHz)[1] |
|---|---|---|---|---|---|
| | Embedded | | Fabric | | Virtex-4 |
| | Type | Number | LUTs | FFs | -10 |
| Add/Subtract | Logic (no usage) | | 388 | 430 | 324 |
| Fixed to float | Int24 input | | 166 | 178 | 349 |
| Float to fixed | Int24 result | | 217 | 188 | 346 |
| Float to float | Single to 24-17 format | | 87 | 89 | 322 |
| | 24-17 to single | | 33 | 52 | 497 |
| Compare | Programmable | | 79 | 24 | 338 |
| Divide | C_RATE=1 | | 460 | 751 | 313 |
| | C_RATE=19 | | 200 | 188 | 290 |
| Sqrt | C_RATE=1 | | 308 | 447 | 325 |
| | C_RATE=18 | | 159 | 158 | 304 |

1. Maximum frequency obtained with map switches -ol high and -cm speed, and par switches -pl high and -rl high.

*Table 23:* **Characterization of 17-Bit Fraction and 24-Bit Total Wordlength on Virtex-5 FPGA**

| Operation | Resources | | | | Maximum Frequency (MHz)[1] |
|---|---|---|---|---|---|
| | Embedded | | Fabric | | Virtex-5 |
| | Type | Number | LUTs | FFs | -1 |
| Multiply | DSP48E (max usage) | 2 | 74 | 133 | 450 |
| | DSP48E (full usage) | 1 | 93 | 129 | 406 |
| | Logic (no usage) | 0 | 340 | 401 | 328 |
| Add/ Subtract | Logic (no usage) | | 341 | 408 | 399 |
| Fixed to float | Int24 input | | 139 | 137 | 355 |
| Float to fixed | Int24 result | | 166 | 188 | 396 |
| Float to Float | Single to 24-17 format | | 74 | 89 | 418 |
| | 24-17 to single | | 35 | 52 | 478 |
| Compare | Programmable | | 66 | 24 | 394 |
| Divide | C_RATE=1 | | 457 | 751 | 394 |
| | C_RATE=19 | | 179 | 189 | 343 |
| Sqrt | C_RATE=1 | | 332 | 447 | 430 |
| | C_RATE=18 | | 135 | 158 | 411 |

1. Maximum frequency obtained with map switches -ol high and -cm speed, and par switches -pl high and -rl high.

### Single-Precision Format

The resource requirements and maximum clock rates achievable with single-precision format on Spartan-3E FPGAs are summarized in Table 24, on Virtex-4 in Table 25, and on Virtex-5 in Table 26.

*Table 24:* **Characterization of Single-Precision Format on Spartan-3E FPGA**

| Operation | Resources | | | | Maximum Frequency (MHz)[1] |
|---|---|---|---|---|---|
| | Embedded | | Fabric | | Spartan-3E |
| | Type | Number | LUTs | FFs | -4 |
| Multiplier | MULT18X18S | 4 | 185 | 275 | 162 |
| | Logic (no usage) | 0 | 630 | 696 | 171 |
| Add/ Subtract | Logic (no usage) | 0 | 580 | 591 | 230 |
| Fixed to float | Int32 input | | 221 | 227 | 198 |
| Float to fixed | Int32 result | | 251 | 237 | 195 |
| Float to float | Single to double | | 15 | 101 | 254 |
| Compare | Programmable | | 100 | 24 | 213 |
| Divide | C_RATE=1 | | 824 | 1,370 | 186 |
| | C_RATE=26 | | 234 | 229 | 159 |
| Sqrt | C_RATE=1 | | 513 | 787 | 189 |
| | C_RATE=25 | | 214 | 206 | 181 |

1. Maximum frequency obtained with map switches -ol high and -cm speed, and par switches -pl high and -rl high.

*Table 25:* **Characterization of Single-Precision Format on Virtex-4 FPGA**

| Operation | Resources | | | | Maximum Frequency (MHz)[1] |
|---|---|---|---|---|---|
| | Embedded | | Fabric | | Virtex-4 |
| | Type | Number | LUTs | FFs | -10 |
| Multiply | DSP48 (max usage) | 5 | 116 | 235 | 391 |
| | DSP48 (full usage) | 4 | 139 | 259 | 353 |
| | DSP48 (medium usage) | 1 | 509 | 562 | 279 |
| | Logic (no usage) | 0 | 641 | 698 | 274 |
| Add/ Subtract | DSP48 (full usage) | 4 | 372 | 466 | 382 |
| | Logic (no usage) | 0 | 578 | 594 | 368 |
| Fixed to float | Int32 input | | 226 | 233 | 318 |
| Float to fixed | Int32 result | | 282 | 238 | 305 |
| Float to float | Single to double | | 43 | 101 | 474 |
| Compare | Programmable | | 97 | 24 | 337 |

*Table 25:* **Characterization of Single-Precision Format on Virtex-4 FPGA *(Continued)***

| Operation | Resources | | | | Maximum Frequency (MHz)[1] |
|---|---|---|---|---|---|
| | Embedded | | Fabric | | Virtex-4 |
| | Type | Number | LUTs | FFs | -10 |
| Divide | C_RATE=1 | | 824 | 1,370 | 278 |
| | C_RATE=26 | | 262 | 229 | 260 |
| Sqrt | C_RATE=1 | | 513 | 787 | 305 |
| | C_RATE=25 | | 213 | 206 | 280 |

1. Maximum frequency obtained with map switches -ol high and -cm speed, and par switches -pl high and -rl high.

*Table 26:* **Characterization of Single-Precision Format on Virtex-5 FPGA**

| Operation | Resources | | | | Maximum Frequency (MHz)[1] |
|---|---|---|---|---|---|
| | Embedded | | Fabric | | Virtex-5 |
| | Type | Number | LUTs | FFs | -1 |
| Multiply | DSP48E (max usage) | 3 | 88 | 177 | 450 |
| | DSP48E (full usage) | 2 | 126 | 209 | 429 |
| | DSP48E (medium usage) | 1 | 294 | 390 | 375 |
| | Logic | 0 | 641 | 698 | 357 |
| Add/Subtract | DSP48E (speed optimized, full usage) | 2 | 267 | 375 | 410 |
| | Logic (speed optimized, no usage) | 0 | 429 | 561 | 395 |
| | Logic (low latency) | 0 | 536 | 625 | 372 |
| Fixed to float | Int32 input | | 181 | 226 | 398 |
| Float to fixed | Int32 result | | 218 | 237 | 373 |
| Float to float | Single to double | | 44 | 101 | 466 |
| Compare | Programmable | | 80 | 24 | 393 |
| Divide | C_RATE=1 | | 788 | 1,370 | 365 |
| | C_RATE=26 | | 227 | 233 | 316 |
| Sqrt | C_RATE=1 | | 542 | 787 | 398 |
| | C_RATE=25 | | 175 | 204 | 388 |

1. Maximum frequency obtained with map switches -ol high and -cm speed, and par switches -pl high and -rl high.

### Double-Precision Format

The resource requirements and maximum clock rates achievable with double-precision format on Spartan-3E FPGAs are summarized in Table 27, on Virtex-4 in Table 28, and on Virtex-5 in Table 29.

*Table 27:* **Characterization of Double-Precision Format on Spartan-3E FPGA**

| Operation | Resources | | | | Maximum Frequency (MHz)[1] |
|---|---|---|---|---|---|
| | Embedded | | Fabric | | Spartan-3E |
| | Type | Number | LUTs | FFs | -4 |
| Multiply | MULT18X18S (full usage) | 16 | 681 | 1,075 | 104 |
| | Logic (no usage) | 0 | 2,296 | 2,451 | 124 |
| Add/ Subtract | Logic (no usage) | 0 | 1,272 | 1,171 | 192 |
| Fixed to float | Int64 input | | 563 | 506 | 103 |
| Float to fixed | Int64 result | | 523 | 446 | 145 |
| Float to float | Double to single | | 95 | 113 | 168 |
| Compare | Programmable | | 164 | 24 | 172 |
| Divide | C_RATE=1 | | 3,335 | 6,002 | 126 |
| | C_RATE=26 | | 437 | 403 | 120 |
| Sqrt | C_RATE=1 | | 1,904 | 3,234 | 132 |
| | C_RATE=25 | | 444 | 393 | 131 |

1. Maximum frequency obtained with map switches -ol high and -cm speed, and par switches -pl high and -rl high.

*Table 28:* **Characterization of Double-Precision Format on Virtex-4 FPGA**

| Operation | Resources | | | | Maximum Frequency (MHz)[1] |
|---|---|---|---|---|---|
| | Embedded | | Fabric | | Virtex-4 |
| | Type | Number | LUTs | FFs | -10 |
| Multiply | DSP48 (max usage) | 17 | 551 | 759 | 381 |
| | DSP48 (full usage) | 16 | 550 | 774 | 303 |
| | DSP48 (medium usage) | 9 | 1,332 | 1,658 | 229 |
| | Logic (no usage) | 0 | 2,311 | 2,457 | 185 |
| Add/ Subtract | DSP48 (full usage) | 3 | 1,220 | 1,139 | 324 |
| | Logic (no usage) | 0 | 1,274 | 1,139 | 284 |
| Fixed to float | Int64 input | | 565 | 506 | 219 |
| Float to fixed | Int64 result | | 523 | 447 | 214 |
| Float to float | Double to single | | 121 | 113 | 310 |
| Compare | Programmable | | 161 | 24 | 246 |

*Table 28:* **Characterization of Double-Precision Format on Virtex-4 FPGA *(Continued)***

| Operation | Resources | | | | Maximum Frequency (MHz)[1] |
|---|---|---|---|---|---|
| | Embedded | | Fabric | | Virtex-4 |
| | Type | Number | LUTs | FFs | -10 |
| Divide | C_RATE=1 | | 3,335 | 6,002 | 192 |
| | C_RATE=26 | | 437 | 401 | 182 |
| Sqrt | C_RATE=1 | | 1,904 | 3,234 | 205 |
| | C_RATE=25 | | 445 | 392 | 201 |

1. Maximum frequency obtained with map switches -ol high and -cm speed, and par switches -pl high and -rl high.

*Table 29:* **Characterization of Double-Precision Format on Virtex-5 FPGA**

| Operation | Resources | | | | Maximum Frequency (MHz)[1] |
|---|---|---|---|---|---|
| | Embedded | | Fabric | | Virtex-5 |
| | Type | Number | LUTs | FFs | -1 |
| Multiply | DSP48E (max usage) | 13 | 416 | 654 | 431 |
| | DSP48E (full usage) | 12 | 424 | 669 | 369 |
| | Logic | 0 | 2,309 | 2,457 | 237 |
| Add/Subtract | DSP48E (speed optimized, full usage) | 3 | 821 | 976 | 356 |
| | Logic (speed optimized, no usage) | 0 | 804 | 1,060 | 316 |
| | Logic (low latency, no usage) | 0 | 1,045 | 1,185 | 291 |
| Fixed to float | Int64 input | | 402 | 504 | 306 |
| Float to fixed | Int64 result | | 396 | 446 | 297 |
| Float to Float | Double to single | | 107 | 131 | 438 |
| Compare | Programmable | | 142 | 24 | 346 |
| Divide | C_RATE=1 | | 3,228 | 6,002 | 254 |
| | C_RATE=26 | | 354 | 399 | 252 |
| Sqrt | C_RATE=1 | | 1,940 | 3,234 | 284 |
| | C_RATE=25 | | 355 | 391 | 278 |

1. Maximum frequency obtained with map switches -ol high and -cm speed, and par switches -pl high and -rl high.

## Ordering Information

This core may be downloaded from the Xilinx IP Center for use with the Xilinx CORE Generator v8.2i. The Xilinx CORE Generator is bundled with the ISE Foundation Series Development software at no additional charge.

Information about additional Xilinx LogiCORE™ modules is available on the Xilinx IP Center or by contacting your local Xilinx sales representative.

## Support

Provided by Xilinx, Inc. @ www.xilinx.com/support.

## References

1. *ANSI/IEEE, IEEE Standard for Binary Floating-Point Arithmetic,* ANSI/IEEE Standard 754-1985. IEEE-754.

## Revision History

This table shows the revision history of this document.

| Date | Version | Revision |
|---|---|---|
| 04/28/05 | 1.0 | Initial Xilinx release. |
| 07/27/05 | 1.1 | Document modified to include minor corrections and section on simulation. |
| 01/18/06 | 2.0 | Updated to version 2.0 of core, Xilinx tools v8.1i. |
| 09/28/06 | 3.0 | Updated to version 3.0 of core, Xilinx tools v8.2i. |